Bengaluru Urban Mobility & Transit Analytics

Multi-Modal (Bike · Auto · Cab) Demand, Congestion, Fare and ETA Analytics using MapReduce and Machine Learning

CA3 — Activity 2: End-to-End Big Data Analytics Pipeline

Course: Big Data Analytics (25MCAAIE3042)

MCA AIML — III Semester

Team Members: [Your Name], [Teammate Name]

Date: [Submission Date]

1. Problem Statement

Bengaluru commuters choose between bike taxis, auto-rickshaws and cabs on a road network that slows sharply in the morning and evening peaks and again whenever it rains. Riders want to know which mode is fastest and cheapest for a given trip and time; drivers want to know where and when demand and earnings are highest; city planners and platform operators need to see where congestion is worst and how demand is spread across hubs and hours.

This project builds an end-to-end Big Data pipeline — data generation, cleaning and feature engineering, MapReduce aggregation, analytics and visualisation, and machine learning — for multi-modal trips across ten Bengaluru hubs, and exposes the results through a role-based web dashboard (User, Driver and Admin views).

2. Dataset

No open, trip-level dataset exists for Bengaluru bike-taxi, auto and cab rides, so the dataset is **synthetic**: it is generated by pipeline/00_generate_bengaluru_data.py from documented city rules, with a fixed random seed (42) so every run produces exactly the same data. All rules live in a single module, pipeline/mobility_model.py, which the web backend also uses, so fares and travel times are consistent between the data, the models and the live predictions.

Property	Value
Raw records (incl. injected dirty rows)	250,750
Raw columns	11 — trip_id, vehicle_type, pickup/dropoff_datetime, pickup/dropoff_zone, distance_km, duration_min, fare_inr, surge_multiplier, is_raining
Records after cleaning	247,951
Date range	1 Jan 2025 – 30 Jun 2025 (181 days, 25 rainy days)
Hubs (real coordinates)	Indiranagar, Koramangala, Whitefield, HSR Layout, Electronic City, MG Road, Hebbal, Kempegowda Airport (BLR), Majestic (KSR Station), Jayanagar
Vehicle types	Bike taxi, auto-rickshaw, cab

2.1 Generation rules

- **Demand:** trips per day × hour follow weekday / weekend profiles; each hub has a type (residential, tech park, mixed, transit) that shapes when it generates pickups. Drop-offs follow a gravity model (nearer, larger hubs are likelier).
- **Traffic:** morning peak 08:30–11:30 (1.5–2.2× slower), evening peak 17:30–21:30 (1.6–2.4×), free-flow at night 22:00–06:00; weekends are milder; each hub has a choke factor; rain adds 15%.
- **Vehicles:** bikes are least affected by congestion but lose the most speed in rain; cabs are slowest at choke points.
- **Fares:** Bike ₹20 (incl. 2 km) + ₹8/km + ₹1.5/delay-min; Auto ₹30 (incl. 1.8 km) + ₹15/km + ₹2/min; Cab ₹60 (incl. 3 km) + ₹18–22/km (rising with traffic) + ₹3/min. Night surcharge 1.5× for autos (Bengaluru meter rule) and 1.2× for bikes and cabs; surge 1.0–2.2×.
- **Mode choice:** a softmax over utilities — bikes favoured for short trips, cabs for long, airport, night and rainy trips.

- **Dirty data:** about 1.5% of rows are deliberately corrupted (duplicates, missing fares and zones, bad distances, inconsistent durations, mis-cased vehicle names) so the cleaning stage has real work to do.

3. Data Cleaning & Preprocessing (Stage 1)

Implemented in pipeline/01_clean_and_engineer.py. Every step and its row count is printed in the run log (Appendix A).

Cleaning step	Rows affected
Exact duplicate rows removed	750
Vehicle names normalised (upper-case / padded with spaces)	600
Rows with missing or unknown pickup zone removed	600
Zero or negative distance removed	600
Duration inconsistent with pickup/drop-off timestamps (> 1 min difference) removed	600
Missing fares imputed from the rate card	600
Extreme outliers removed (duration > 99.9th percentile, 185 min)	249

Engineered features (30 columns in total): hour, day_of_week, day_name, month, is_weekend, is_peak_hour, is_night, time_period, day_type, traffic_factor, traffic_level, free_flow_min, delay_min, avg_speed_kmph, corridor (pickup → drop-off), distance_category (Short < 3 km, Medium 3–8, Long 8–15, Very Long > 15), fare_per_km and fare_per_min.

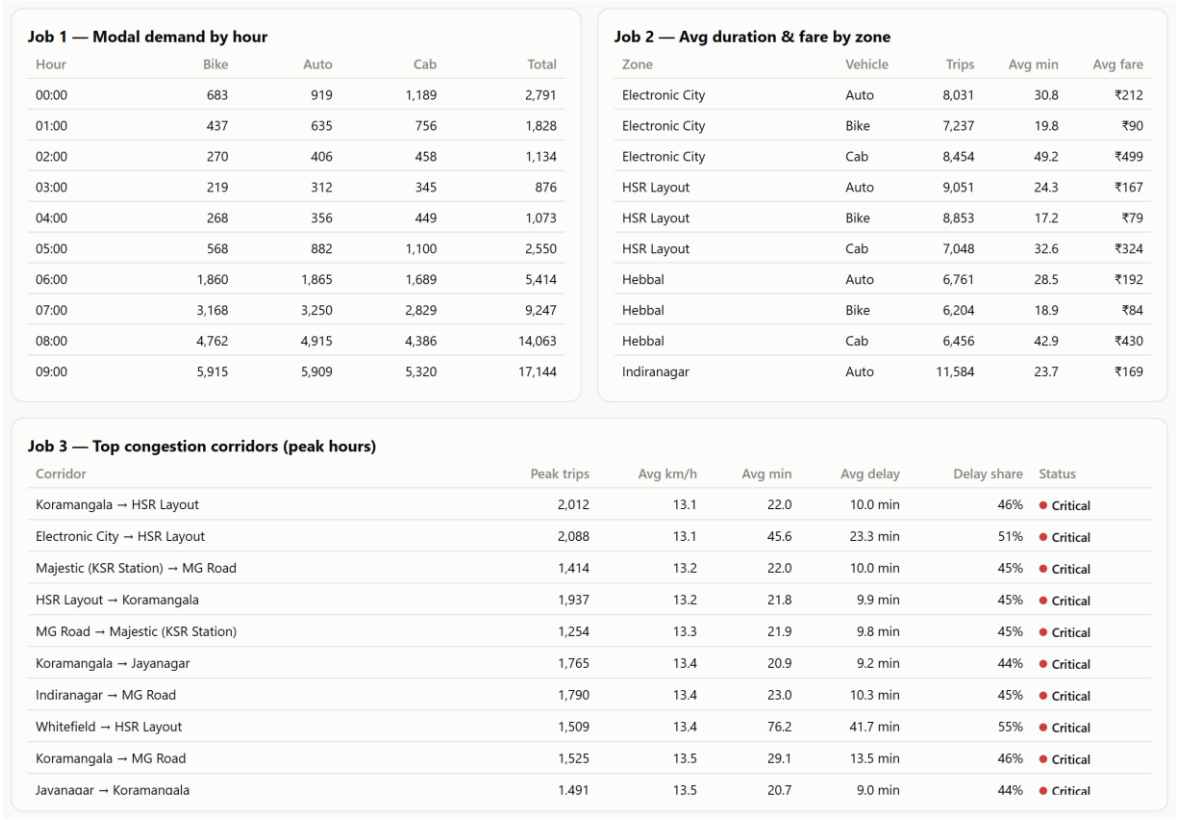
Net result: 247,951 clean records, written to outputs/cleaned_bengaluru_mobility.csv, plus a 12-column tab-separated file (outputs/mapreduce_input.tsv) whose column order is fixed in pipeline/schema.py and shared by every mapper.

4. MapReduce Implementation (Stage 2)

Each job is a Hadoop-Streaming-compatible mapper.py / reducer.py pair in pipeline/mapreduce/job*/. pipeline/02_run_mapreduce.py runs each one as Map → Shuffle/Sort → Reduce: the mapper reads the TSV on stdin and writes key<TAB>value lines, the output is sorted by key, and the reducer aggregates each key group. This is the same contract Hadoop Streaming uses, so the files would run unchanged with “hadoop jar hadoop-streaming.jar -mapper mapper.py -reducer reducer.py …”. The jobs were run locally, without a Hadoop cluster.

Job	Map: key → value	Reduce output	Rows
1 · Modal demand by hour	(hour | vehicle) → 1	trip count	72
2 · Duration & fare by zone	(pickup_zone | vehicle) → duration, fare	count, avg duration, avg fare, total fare	30
3 · Top congestion corridors	peak-hour, inter-zone trips only: (pickup | drop-off) → km, minutes, delay	trips, avg speed, avg duration, avg delay, delay share, status	90
4 · Demand dataset	(date | hour | day | zone | is_raining) → 1	trip count per cell (feeds Model B)	38,158

Job 3 labels a corridor CRITICAL below 13.75 km/h and SEVERE below 15 km/h (thresholds calibrated to this data's speed spread), giving 21 Critical, 46 Severe and 23 Moderate corridors. All four jobs process 247,951 records in about 1.5 seconds locally (Job 3 maps the 64,762 peak, inter-zone trips).



Job 1 — Modal demand by hour

Hour	Bike	Auto	Cab	Total
00:00	683	919	1,189	2,791
01:00	437	635	756	1,828
02:00	270	406	458	1,134
03:00	219	312	345	876
04:00	268	356	449	1,073
05:00	568	882	1,100	2,550
06:00	1,860	1,865	1,689	5,414
07:00	3,168	3,250	2,829	9,247
08:00	4,762	4,915	4,386	14,063
09:00	5,915	5,909	5,320	17,144

Job 2 — Avg duration & fare by zone

Zone	Vehicle	Trips	Avg min	Avg fare
Electronic City	Auto	8,031	30.8	₹212
Electronic City	Bike	7,237	19.8	₹90
Electronic City	Cab	8,454	49.2	₹499
HSR Layout	Auto	9,051	24.3	₹167
HSR Layout	Bike	8,853	17.2	₹79
HSR Layout	Cab	7,048	32.6	₹324
Hebbal	Auto	6,761	28.5	₹192
Hebbal	Bike	6,204	18.9	₹84
Hebbal	Cab	6,456	42.9	₹430
Indiranagar	Auto	11,584	23.7	₹169

Job 3 — Top congestion corridors (peak hours)

Corridor	Peak trips	Avg km/h	Avg min	Avg delay	Delay share	Status
Koramangala → HSR Layout	2,012	13.1	22.0	10.0 min	46%	Critical
Electronic City → HSR Layout	2,088	13.1	45.6	23.3 min	51%	Critical
Majestic (KSR Station) → MG Road	1,414	13.2	22.0	10.0 min	45%	Critical
HSR Layout → Koramangala	1,937	13.2	21.8	9.9 min	45%	Critical
MG Road → Majestic (KSR Station)	1,254	13.3	21.9	9.8 min	45%	Critical
Koramangala → Jayanagar	1,765	13.4	20.9	9.2 min	44%	Critical
Indiranagar → MG Road	1,790	13.4	23.0	10.3 min	45%	Critical
Whitefield → HSR Layout	1,509	13.4	76.2	41.7 min	55%	Critical
Koramangala → MG Road	1,525	13.5	29.1	13.5 min	46%	Critical
Jayanagar → Koramangala	1,491	13.5	20.7	9.0 min	44%	Critical

Figure 1. Admin view — raw output of MapReduce Jobs 1, 2 and 3, as shown in the dashboard.

5. Data Analytics & Visualisation (Stage 3)

pipeline/04_precompute_dashboard.py turns the cleaned data and MapReduce outputs into one JSON file split by audience (common, user, driver, admin). The dashboard draws every chart from it with Chart.js, in light and dark themes.

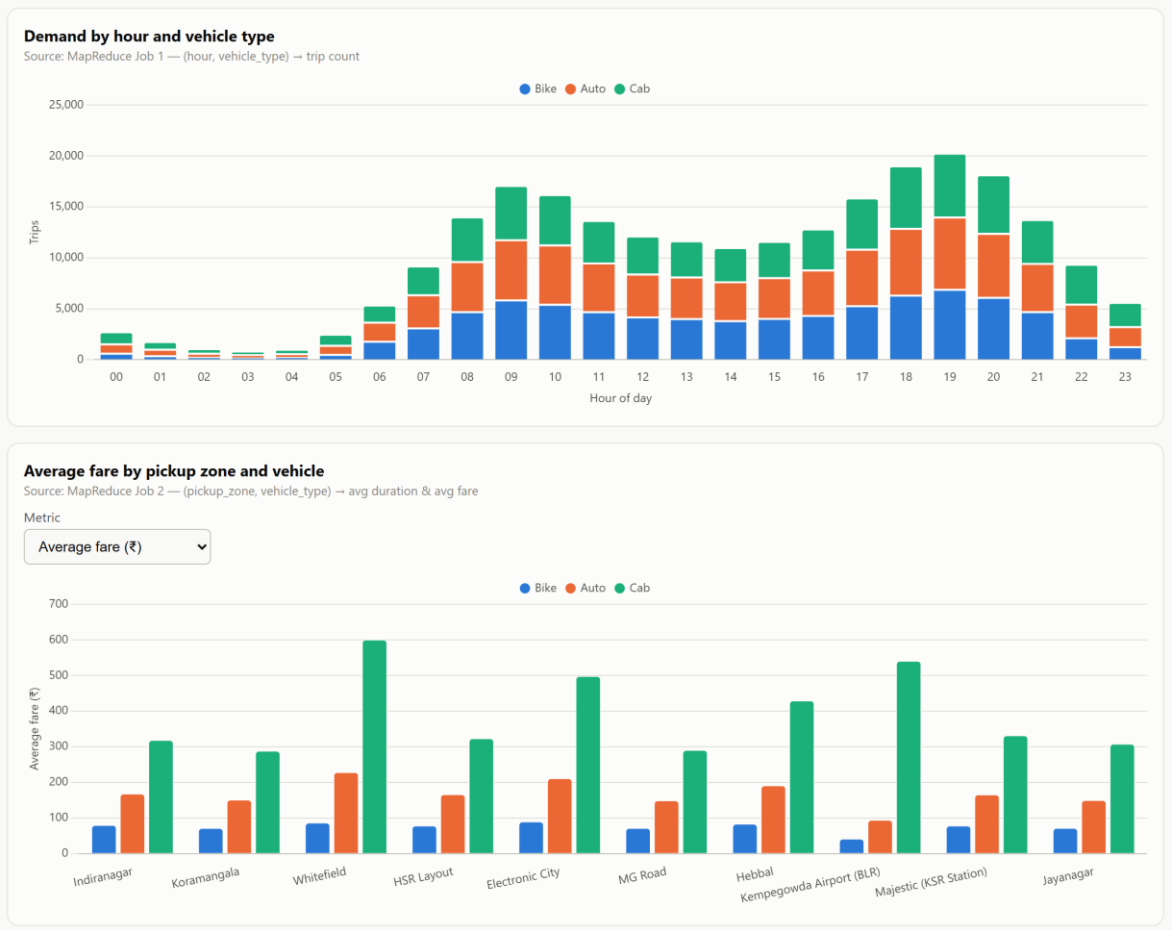


Figure 2. Demand by hour and vehicle type (Job 1) and average fare by pickup zone and vehicle (Job 2).

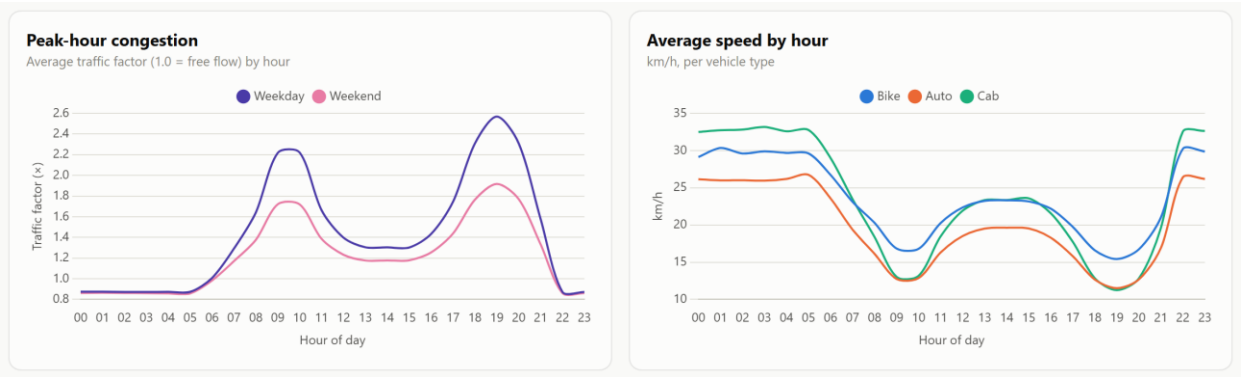


Figure 3. Traffic factor by hour for weekdays and weekends, and average speed by hour per vehicle.

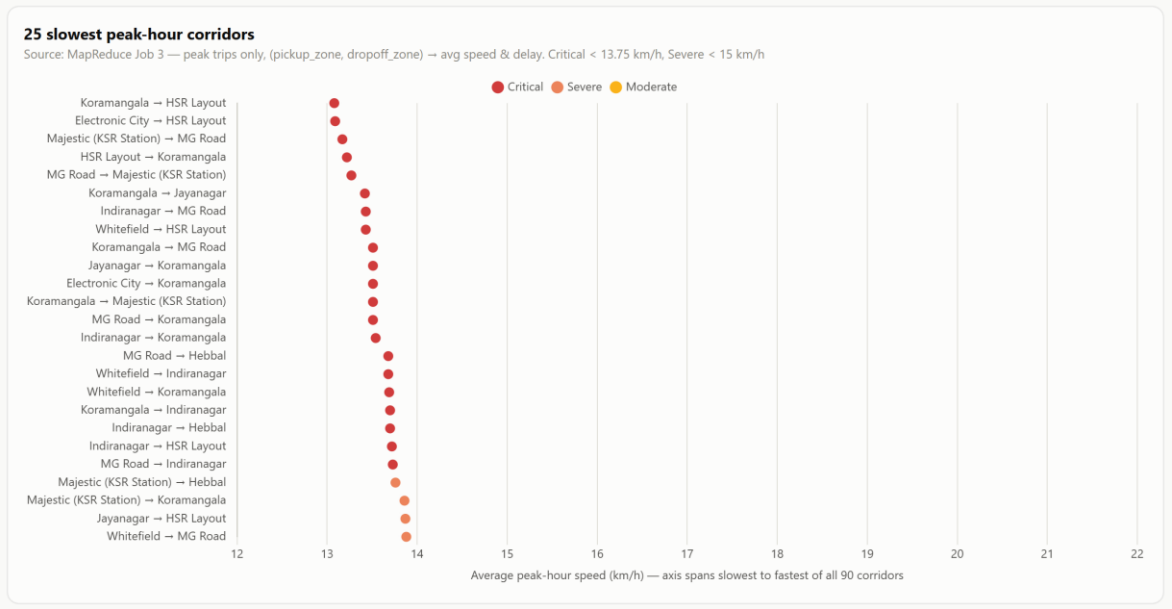


Figure 4. The 25 slowest peak-hour corridors from Job 3, coloured by status.

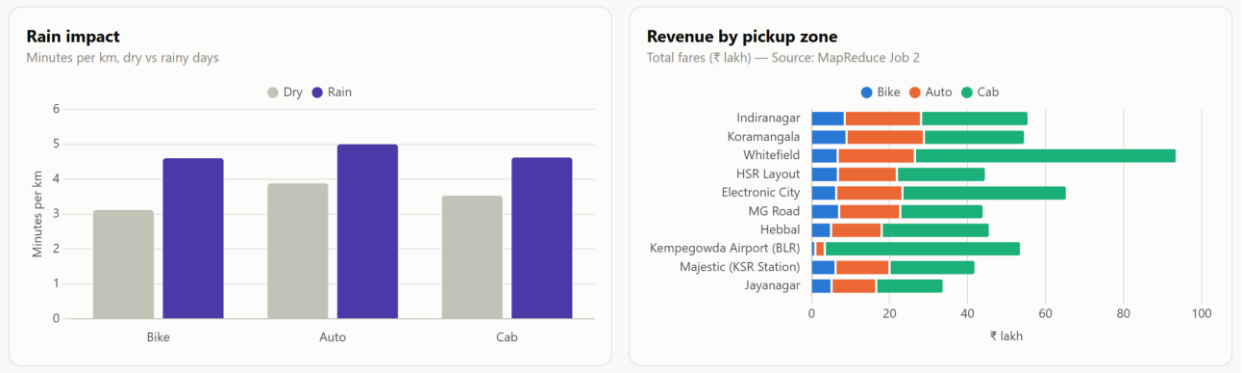


Figure 5. Rain impact on minutes per km, and total fare value by pickup zone.

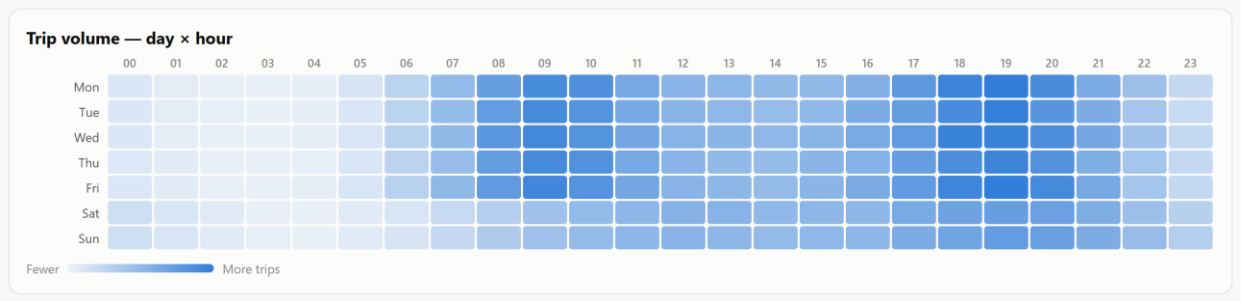


Figure 6. Trip volume by day of week and hour.

5.1 Key findings

Metric	Value
Total trips (after cleaning)	247,951
Total fare value	₹5.36 crore
Average fare / duration / distance	₹216 / 26.9 min / 7.78 km
Average speed / average surge	17.4 km/h / 1.39×
Mode share (bike / auto / cab)	33.2% / 34.7% / 32.1%
Share of fare value (bike / auto / cab)	12% / 28% / 61%

Metric	Value
Peak demand hour / busiest day / busiest hub	19:00 / Wednesday / Koramangala
Trips in the two peak windows	48.3%
Slowest corridor (peak)	Koramangala → HSR Layout, 13.1 km/h
Distance ↔ duration correlation	0.85

- Demand has two clear peaks, 09:00–10:00 and 18:00–20:00, with the overall maximum at 19:00; it is lowest from 02:00 to 04:00. Weekdays carry 76% of all trips.
- Peak traffic slows every mode, but not equally: off-peak to peak, bikes fall from 23.0 to 16.8 km/h, autos from 19.5 to 12.8 and cabs from 23.3 to 13.1.
- Rain hurts bikes most: minutes per km rise 47% for bikes against 31% for cabs.
- Cabs are 32% of trips but about 61% of fare value, because cab trips are longer (11.5 km on average vs 5.5 km for bikes) and pricier per km.
- The slowest corridors form two clusters: Koramangala / HSR Layout / Electronic City (the Silk Board junction area) and Majestic ↔ MG Road in the old city centre.
- Drivers earn less per trip-minute in peak jams (autos about ₹6.5 per minute by day vs ₹11.8 at night): delay charges do not keep pace with time lost in traffic.
- At night an auto costs as much as a cab for trips of 8 km or more (₹184 vs ₹180 for 8 km at 23:00), because of the 1.5× night meter; in the afternoon the same auto trip is ₹131 against ₹167 by cab.

6. Machine Learning (Stage 4)

pipeline/03_train_models.py trains two Random Forest models (80 trees, maximum depth 12, at least 5 samples per leaf, random_state 42) with an 80/20 train/test split, and saves them with joblib compression (1.6 MB and 3.8 MB) together with their metrics as JSON.

Model A — Trip duration / ETA (regression)

Features: vehicle type, distance (km), hour, day of week, traffic factor and is_raining. Rain was added to the brief's feature list so the model can learn that bikes slow down most in rain.

Target: instead of raw minutes, the model learns the ratio of the actual duration to a formula-based ETA (from mobility_model.expected_duration_minutes), and the prediction is that ratio × the formula ETA. With a raw-minutes target, long trips that are rare for bikes and autos (such as a 52 km night trip to the airport) all received the same ETA, because decision trees cannot extrapolate beyond the training range. The fare is not modelled: it is computed from the predicted ETA with the rate card.

Metric	Value
Mean absolute error (all vehicles)	2.995 min
R^2 score	0.9573
MAE — bike / auto / cab	1.875 / 2.767 / 4.383 min
Formula-only baseline MAE	2.99 min
Top features (importance)	distance 42%, traffic factor 34%, hour 10%, day of week 9%
Training rows	247,951

Honest caveat: the formula alone scores the same MAE as the model, because the synthetic durations were generated from that formula plus random noise. On real trip data the model would be expected to add value over the formula; on this data it mainly confirms the formula.

Model B — Zone demand level → surge (classification)

Trained on the 38,158 date × hour × zone cells produced by MapReduce Job 4. Each cell is labelled Low (≤ 4 trips), Medium (≤ 8) or High using the 33rd and 66th percentiles. Features: hour, day of week, is_weekend, is_peak_hour, zone and is_raining. The predicted level sets the surge multiplier: Low 1.0×, Medium 1.1×, High 1.3×, plus 0.2 in peak hours and 0.3 in rain, capped at 2.2×.

Metric	Value
Accuracy	68.0% (random guessing: 33%)
Weighted F1-score	0.6873
Class balance	Low 15,442 / Medium 12,220 / High 10,496

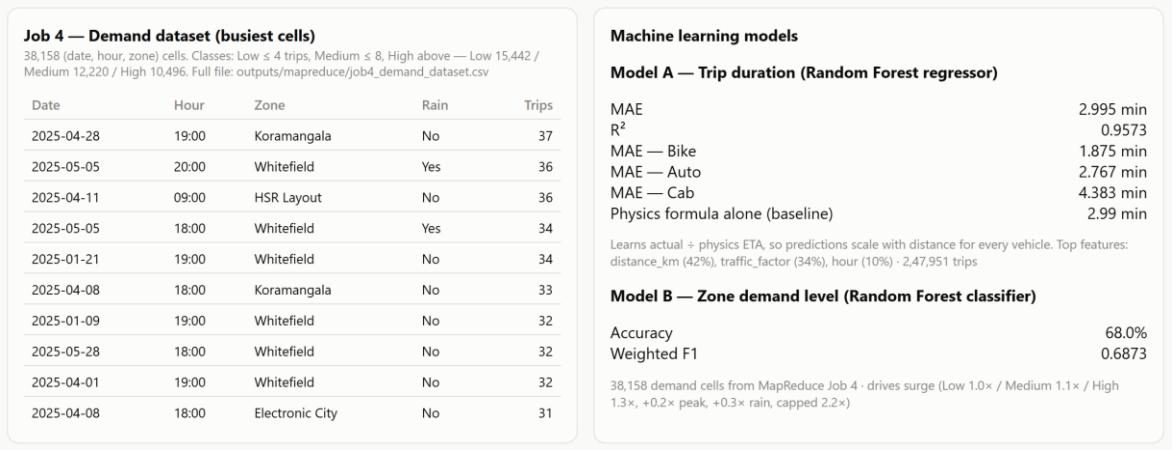


Figure 7. Admin view — Job 4 output (busiest cells) and the metrics card for both models.

7. Web Dashboard & API

backend/app.py (Flask) loads the precomputed JSON and both models at start-up and serves:

Endpoint	Purpose
GET /api/dashboard?role=user|driver|admin	Precomputed data for one role
POST /api/predict/trip	ETA, fare breakdown and surge for bike, auto and cab between two hubs
POST /api/predict/demand	Demand level (Low / Medium / High), class probabilities and surge for a hub
GET /api/health	Readiness check for the data and models

Invalid input (unknown role or zone, hour outside 0–23, distance outside 0.3–80 km, malformed JSON) returns a 400 error with a message. Run with “python backend\app.py” and open http://127.0.0.1:5000.

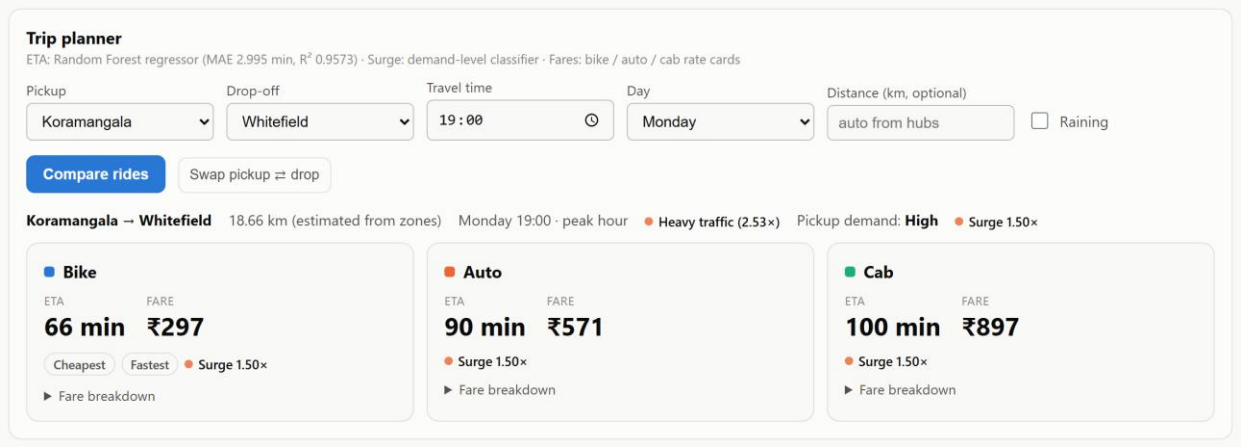


Figure 8. User view — trip planner comparing all three modes for Koramangala → Whitefield, Monday 19:00.

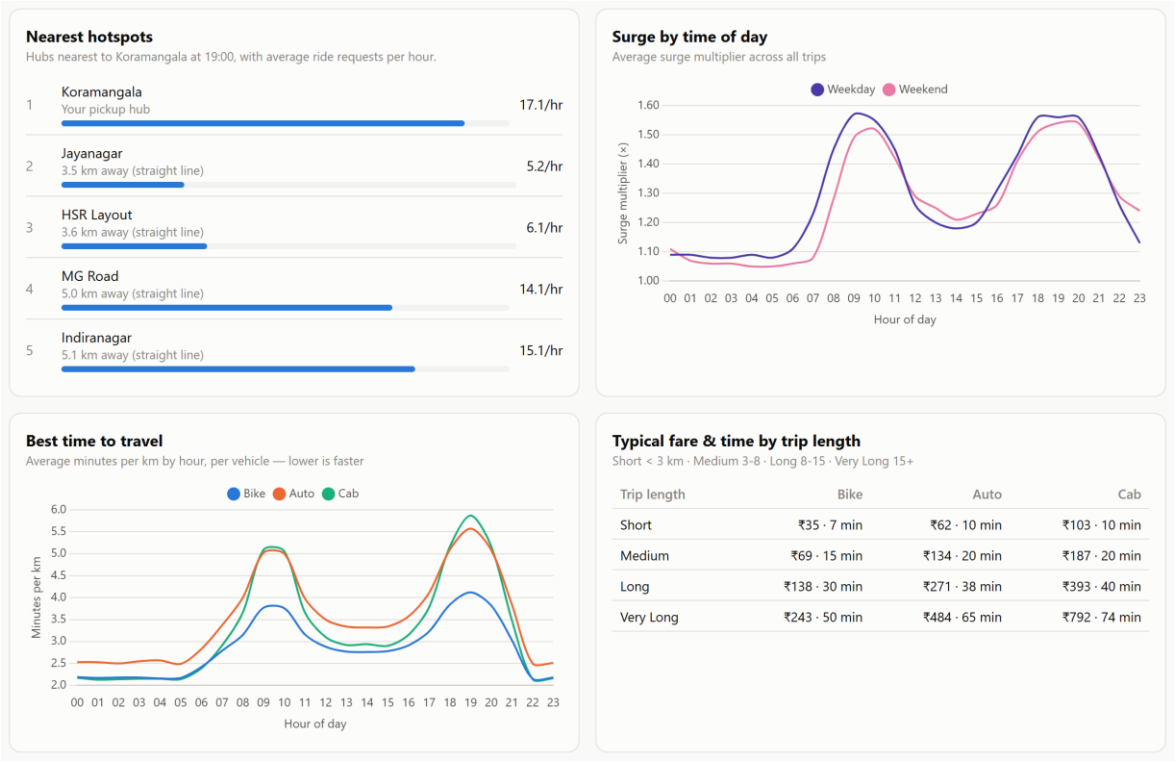


Figure 9. User view — nearby hotspots, surge by time of day, best time to travel and typical fares by trip length.

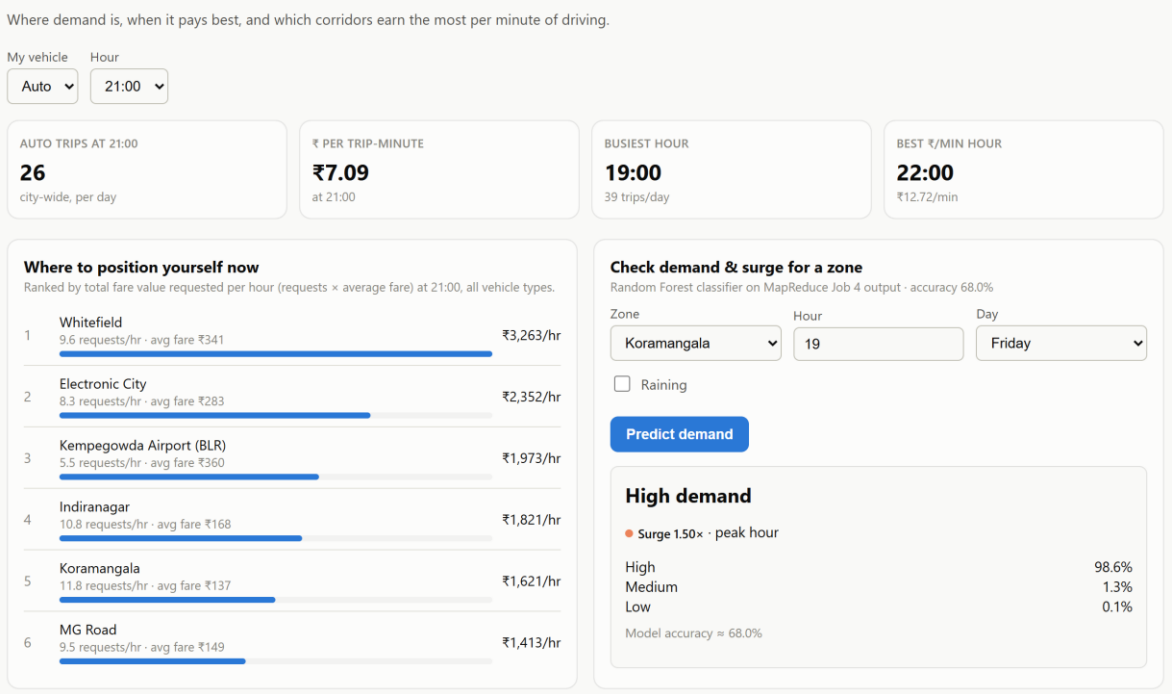


Figure 10. Driver view — where to wait now and the live demand & surge check (Model B).

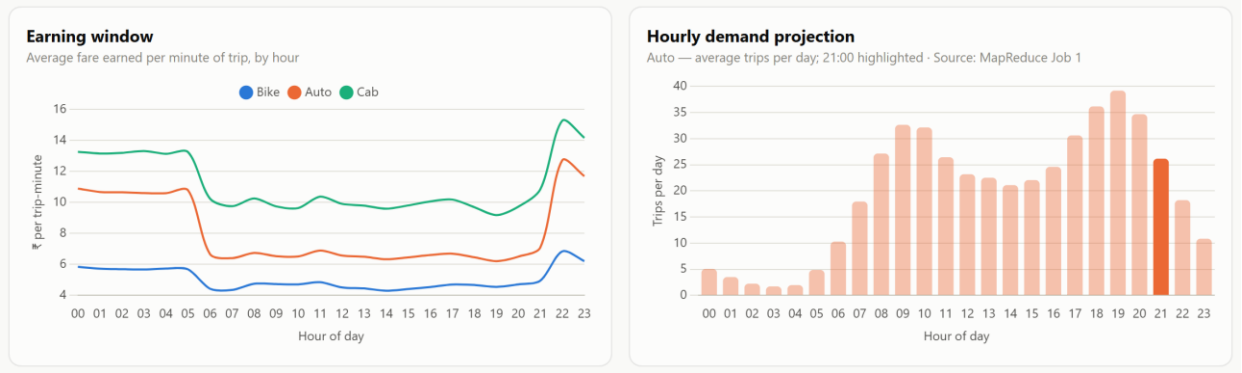


Figure 11. Driver view — fare earned per trip-minute by hour, and hourly demand projection.

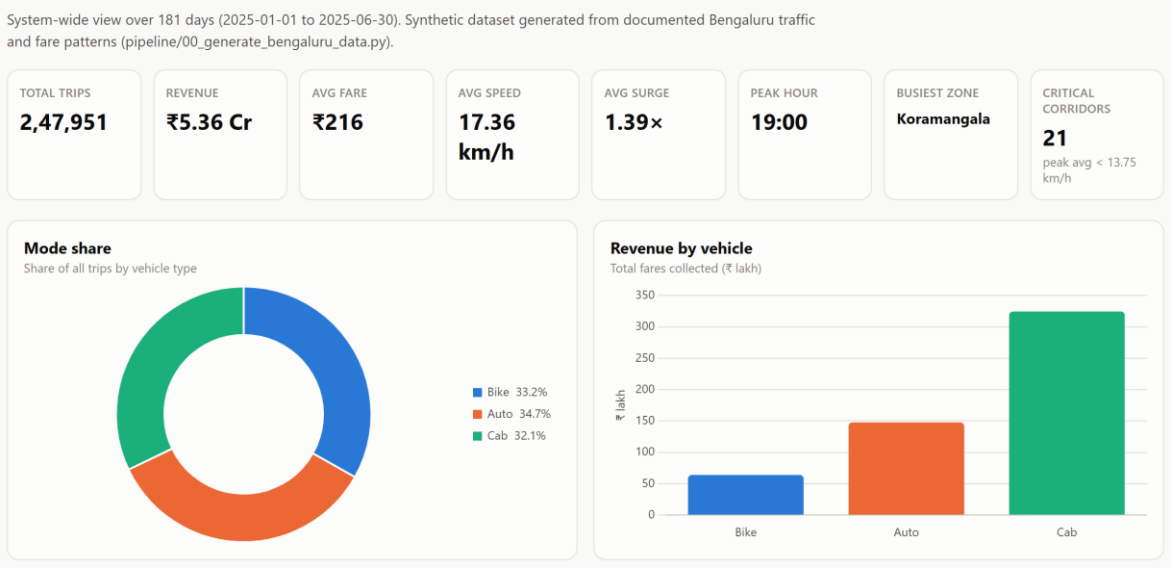


Figure 12. Admin view — system KPIs, mode share and fare value by vehicle.

8. Insights & Recommendations

- **Put supply where the peaks are.** Shift drivers towards 09:00–10:00 and 18:00–20:00 (MapReduce Job 1) and away from 02:00–04:00.
- **Fix two corridor clusters first.** Koramangala / HSR Layout / Electronic City and Majestic ↔ MG Road are the slowest peak-hour corridors (13.1–13.3 km/h); they are the best candidates for signal priority, bus lanes or staggered office hours.
- **Recommend the mode by context.** Bikes are the fastest option for short peak-hour trips; cabs are the better choice in rain, when bike travel times rise most. The trip planner already shows this comparison.
- **Review peak-hour driver pay.** Earnings per trip-minute are lowest in traffic jams; a higher per-minute delay rate would keep drivers on the road when they are most needed.
- **Revisit the night auto surcharge.** At 1.5× the auto meter makes autos cost as much as cabs for longer night trips, which pushes riders to cabs.
- **Use the demand model for positioning.** At 68% accuracy across three classes, Model B is good enough to flag likely High-demand hub-hours in advance, and its output already drives the surge shown to riders.

9. Limitations

- **Synthetic data.** The trips are generated from documented rules, not collected from operators, so the findings describe the model of the city rather than measured behaviour. Real operator or city data would be needed to validate them.
- **ML adds little over the formula on this data,** for the reason given in Section 6.
- **MapReduce is simulated locally.** The jobs follow the Hadoop Streaming contract but were not run on a Hadoop cluster.
- **Demand model accuracy (68%)** is limited by the available features; weather forecasts, events and holidays would improve it.

10. Conclusion

The project implements a complete Big Data pipeline for Bengaluru multi-modal mobility: reproducible data generation, cleaning and feature engineering, four Hadoop-Streaming-compatible MapReduce jobs, more than twenty charts and tables, and two machine learning models (regression and classification) served live through a role-based Flask dashboard. The full pipeline processes 247,951 trips end-to-end in about 10 seconds and answers practical questions for riders (which mode, how long, how much), drivers (where and when to work) and planners (where congestion is worst).

Appendix A — Full Pipeline Run Output

Terminal output of “python pipeline\run_all.py”, showing data generation, every cleaning step, all four MapReduce jobs (map / shuffle-sort / reduce timings) and both model training runs:

```
######################################################################
# RUNNING 00_generate_bengaluru_data.py
######################################################################
======================================================================
STAGE 0: GENERATE SYNTHETIC BENGALURU MULTI-MODAL TRIPS
======================================================================
Days simulated: 181 (2025-01-01 .. 2025-06-30), rainy days: 25
Raw rows written: 250,750 (incl. injected dirty/duplicate rows)
Mode shares: {'auto': 0.347, 'bike': 0.331, 'cab': 0.322}
Saved: C:\all\3 sem\bda\a2\outputs\bengaluru_mobility_raw.csv

Stage 0 completed in 1.2s

######################################################################
# RUNNING 01_clean_and_engineer.py
######################################################################
======================================================================
STAGE 1: DATA CLEANING & FEATURE ENGINEERING
======================================================================
Loaded raw data: 250,750 rows, 11 columns
Duplicates removed: 750
Vehicle names normalised (case/whitespace); unknown types removed: 0
Rows with missing/unknown zones removed: 600
Invalid datetime rows removed: 0
Zero/negative distance rows removed: 600
Durations inconsistent with timestamps removed: 600
Missing fares imputed from rate card: 600
Extreme outlier rows removed (>99.9th pct duration, 185 min): 249

Final cleaned dataset: 247,951 rows, 30 columns
Saved: C:\all\3 sem\bda\a2\outputs\cleaned_bengaluru_mobility.csv
Saved MapReduce input: C:\all\3 sem\bda\a2\outputs\mapreduce_input.tsv

Stage 1 completed in 3.2s

######################################################################
# RUNNING 02_run_mapreduce.py
######################################################################
======================================================================
STAGE 2: MAPREDUCE EXECUTION (4 jobs)
======================================================================

======================================================================
MAPREDUCE job1_modal_demand_by_hour
======================================================================
  [1/3] MAP phase ...
        mapped records: 247,951  (0.2s)
  [2/3] SHUFFLE / SORT phase ...
        sorted in 0.1s
  [3/3] REDUCE phase ...
        reduced groups: 72  (0.1s)
  Saved: C:\all\3 sem\bda\a2\outputs\mapreduce\job1_modal_demand_by_hour.csv  (72 rows, total 0.3s)

======================================================================
MAPREDUCE job2_avg_duration_and_fare_by_zone
======================================================================
  [1/3] MAP phase ...
        mapped records: 247,951  (0.3s)
  [2/3] SHUFFLE / SORT phase ...
        sorted in 0.1s
  [3/3] REDUCE phase ...
        reduced groups: 30  (0.1s)
  Saved: C:\all\3 sem\bda\a2\outputs\mapreduce\job2_avg_duration_and_fare_by_zone.csv  (30 rows, total
0.5s)

======================================================================
MAPREDUCE job3_top_congestion_corridors
======================================================================
  [1/3] MAP phase ...
        mapped records: 64,762  (0.1s)
  [2/3] SHUFFLE / SORT phase ...
```

```
        sorted in 0.0s
  [3/3] REDUCE phase ...
        reduced groups: 90  (0.1s)
  Saved: C:\all\3 sem\bda\a2\outputs\mapreduce\job3_top_congestion_corridors.csv  (90 rows, total 0.2s)

======================================================================
MAPREDUCE job4_demand_dataset
======================================================================
  [1/3] MAP phase ...
        mapped records: 247,951  (0.3s)
  [2/3] SHUFFLE / SORT phase ...
        sorted in 0.1s
  [3/3] REDUCE phase ...
        reduced groups: 38,158  (0.1s)
  Saved: C:\all\3 sem\bda\a2\outputs\mapreduce\job4_demand_dataset.csv  (38,158 rows, total 0.5s)

All MapReduce jobs completed in 1.5s

######################################################################
# RUNNING 03_train_models.py
######################################################################
======================================================================
MODEL A: TRIP DURATION PREDICTION (Regression, all 3 vehicle types)
======================================================================
Training rows: 247,951
Trained in 1.1s
MAE: 2.99 min   R2: 0.9573   (physics formula alone: MAE 2.99 min)
MAE by vehicle: {'bike': 1.875, 'auto': 2.767, 'cab': 4.383}

======================================================================
MODEL B: ZONE DEMAND LEVEL / SURGE PREDICTION (Classification)
======================================================================
Demand cells (from MapReduce Job 4): 38,158
Demand level distribution:
demand_level
Low       15442
Medium    12220
High      10496
Name: count, dtype: int64
Trained in 0.1s
Accuracy: 0.6803   Weighted F1: 0.6873

======================================================================
STAGE 3 COMPLETE
======================================================================
{
  "duration_model": {
    "features": [
      "vehicle_type_encoded",
      "distance_km",
      "hour",
      "day_of_week",
      "traffic_factor",
      "is_raining"
    ],
    "target": "duration_min / physics_eta (prediction = model output x
mobility_model.expected_duration_minutes)",
    "vehicle_encoding": {
      "bike": 0,
      "auto": 1,
      "cab": 2
    },
    "mae_minutes": 2.995,
    "physics_only_mae_minutes": 2.99,
    "mae_minutes_by_vehicle": {
      "bike": 1.875,
      "auto": 2.767,
      "cab": 4.383
    },
    "r2_score": 0.9573,
    "feature_importance": {
      "vehicle_type_encoded": 0.0378,
      "distance_km": 0.4157,
      "hour": 0.1006,
      "day_of_week": 0.0949,
      "traffic_factor": 0.3372,
      "is_raining": 0.0138
    },
    "training_rows": 247951
  },
  "demand_model": {
```

```
    "features": [
      "hour",
      "day_of_week",
      "is_weekend",
      "is_peak_hour",
      "zone_code",
      "is_raining"
    ],
    "zone_encoding": {
      "Indiranagar": 0,
      "Koramangala": 1,
      "Whitefield": 2,
      "HSR Layout": 3,
      "Electronic City": 4,
      "MG Road": 5,
      "Hebbal": 6,
      "Kempegowda Airport (BLR)": 7,
      "Majestic (KSR Station)": 8,
      "Jayanagar": 9
    },
    "accuracy": 0.6803,
    "f1_weighted": 0.6873,
    "thresholds": {
      "q33": 4.0,
      "q66": 8.0
    },
    "class_counts": {
      "Low": 15442,
      "Medium": 12220,
      "High": 10496
    },
    "training_rows": 38158
  }
}
```


```
######################################################################
# RUNNING 04_precompute_dashboard.py
######################################################################
Loaded cleaned dataset: 247,951 rows
Saved: C:\all\3 sem\bda\a2\outputs\dashboard_data.json (64 KB)

FULL PIPELINE COMPLETE in 10.0s
```